



Bahria University, Islamabad

Department of Software Engineering

Data Structures and Algorithms lab (Spring-2026)

Instructor: Engr. Faisal Zia

Group Members:

Muhammad Fahad Khan

(09-131242-054)

Ahmed Minhal

(09-131242-010)

Ameer Hamza

(09-131242-018)

Project Report

Submitted: May 2026

Comments:

Signature

UNIVERSITY DSA PROJECT

Online Voting System

PROJECT REPORT

4th Semester | Data Structures and Algorithms Lab

Github-Repository: <https://github.com/fahadk06/DSA-Voting-System-Project>

1. Introduction

Elections are the cornerstone of democratic governance. Traditional voting methods — relying on physical polling stations, paper ballots, and manual counting — are susceptible to delays, human error, logistical challenges, and a lack of real-time transparency. As the scale of elections grows, so does the complexity of managing voter records, candidate data, and vote tallying.

This project presents the Online Voting System, a full-featured desktop application developed in Java Swing (IntelliJ IDEA) that digitalizes the entire election workflow. The system allows voters to register and cast votes electronically, while administrators can manage candidates and monitor live election progress. The application is backed by a MySQL database for persistent data storage and applies core Data Structures and Algorithms to optimize all critical operations.

The project directly addresses the requirements of the DSA Lab Group Project by demonstrating: a Linked List for sequential candidate navigation, Binary Search for efficient candidate retrieval, Bubble Sort (with Insertion Sort as a preprocessing step) for vote ranking, and full CRUD operations connected to a relational database.

2. Problem Statement

Traditional election systems face several well-documented problems:

- Manual counting is slow, error-prone, and difficult to audit.
- Physical voter registries are hard to manage at scale and lack real-time verification.
- Candidate information is not easily searchable or browsable by voters.
- Vote tallying and result ranking require significant human effort and introduce latency.
- Lack of a centralized, persistent data store means records can be lost or tampered with.

The core problem this project addresses is: How can we design a secure, efficient, and user-friendly digital voting system that applies data structures and algorithms to optimize voter management, candidate retrieval, and vote counting — while ensuring all records persist reliably across sessions?

3. Scope of the Project

The Online Voting System covers the following scope:

- Digital voter registration with admin verification workflow.
- Secure role-based login for Administrators and Voters.
- Full candidate management: Add, Update, Delete, Search, and Browse.
- Linked List-based sequential candidate navigation (Next / Previous).
- Binary Search for name-based candidate retrieval.
- Bubble Sort for real-time vote ranking and results display.
- MySQL database for persistent storage of voters, candidates, and vote records.
- Java Swing GUI with a modern dark-themed interface (IntelliJ IDEA).
- GitHub version control with individual branch contributions.
- Support for large datasets (1000+ voter and candidate records).

4. System Architecture and Design

The application follows a layered architecture with three distinct tiers:

Layer	Technology	Responsibility
Presentation Layer	Java Swing (IntelliJ IDEA)	GUI forms, user interaction, event handling
Business / DSA Layer	Pure Java Classes	Linked List, Binary Search, Bubble Sort, Insertion Sort
Data Layer	MySQL via JDBC Connector/J	Persistent storage of voters, candidates, votes

4.1 Package Structure

- `com.votingsystem.gui` — All Swing GUI forms (MainForm, AdminDashboard, UserDashboard, etc.)
- `com.votingsystem.dsa` — Custom DSA implementations (CandidateLinkedList, BinarySearch, VoteSorter)
- `com.votingsystem.database` — Database connectivity (DBConnection singleton)
- `com.votingsystem.model` — Data model classes (Candidate, Voter)

4.2 System Flow Overview

The application entry point is MainForm, which presents a role selector (Administrator / Voter):

Admin Flow

- Admin authenticates via AdminLogin form.
- AdminDashboard loads all candidates from MySQL into both a 2D array and a CandidateLinkedList.
- Admin can Add, Update, Delete, Search (Binary Search), and Browse (Linked List Next/Prev) candidates.
- Admin verifies newly registered voters before granting voting access.
- Admin can view live vote progress and sorted results (Bubble Sort applied before display).

User Flow

- New users register via UserRegistration form (data saved to MySQL).
- After admin verification, users log in via UserLogin form.
- UserDashboard loads candidate list from MySQL, populates Linked List for navigation.
- Users browse candidates (Next/Previous via Linked List), search by area or name (Binary Search).
- User casts a vote; system updates vote_count in MySQL and records the vote.
- VoteProgress screen displays candidates sorted by vote count (Bubble Sort).

5. Features

GUI Features (Java Swing)

- Dark-themed modern interface using custom color palette (DARK_BG, PANEL_BG, BUTTON_BLUE, etc.).
- Segoe UI font throughout for professional readability.
- JTable with alternating row colors, custom renderers, and selection highlighting.
- JComboBox role selector on MainForm for Admin / Voter routing.
- Status bar with live feedback on all operations.
- Responsive window sizing with BorderLayout, GridLayout, and BoxLayout combinations.

6. DSA Design Overview

6.1 Linear Data Structure — Singly Linked List

File: `com/votingsystem/dsa/CandidateLinkedList.java`

A custom Singly Linked List stores candidate records as Node objects. Each Node holds: id (int), name (String), party (String), area (String), votes (int), and a next pointer.

Operation / Method	Description
<code>add(id, name, party, area, votes)</code>	Appends a new Node to the tail of the list.
<code>clear()</code>	Resets head to null and size to 0 for full reload.
<code>nextIndex(current)</code>	Returns $(current + 1) \% size$ — circular forward navigation.
<code>prevIndex(current)</code>	Returns $(current - 1 + size) \% size$ — circular backward navigation.
<code>size()</code>	Returns current number of nodes.

6.2 Non-Linear Data Structure — Binary Search Tree (via Binary Search)

File: `com/votingsystem/dsa/BinarySearch.java`

The BinarySearch class implements a combined Binary Search + Linear Fallback strategy. Before searching, the candidates array is sorted alphabetically by name using Insertion Sort (`sortByName` method). Binary Search then locates name-matching records in $O(\log n)$ time. A linear scan over all records handles area-based queries as a fallback.

Search Type	Algorithm
Search by Candidate Name	Binary Search on sorted array
Search by Area	Linear Scan (fallback)
Pre-sort for Binary Search	Insertion Sort (<code>sortByName</code>)

6.3 Sorting Algorithms — Bubble Sort & Insertion Sort (Comparison)

File: `com/votingsystem/dsa/VoteSorter.java`

Two sorting algorithms are implemented and compared for different use cases:

Algorithm	Use Case
Bubble Sort	Sort candidates by vote count (descending) for results display
Insertion Sort	Pre-sort by name before Binary Search

7. Data Storage Approach

7.1 MySQL Database (Primary Storage)

File: `com/votingsystem/database/DBConnection.java`

The system uses MySQL as its primary persistent data store, connected via the JDBC MySQL Connector/J driver. A `DBConnection` singleton class manages the single shared `Connection` object across the application lifecycle, preventing redundant connections and ensuring consistent transaction management.

Parameter	Value
Database URL	<code>jdbc:mysql://localhost:3306/dsa-project</code>
Driver Class	<code>com.mysql.cj.jdbc.Driver</code> (MySQL Connector/J 8+)
Connection Mode	Singleton — one shared connection per application session
Tables	<code>candidates</code> , <code>voters</code> , <code>votes</code> (relational schema)
Dataset Scale	Supports 1000+ records per table

8. Functional Requirements

8.1 Admin Functions

- FR-A1: Admin must authenticate with a valid username and password stored in MySQL.
- FR-A2: Admin Dashboard must display total candidates, total votes cast, and current leading candidate.
- FR-A3: Admin must be able to add a new candidate (name, party, area) to the MySQL candidates table.
- FR-A4: Admin must be able to update existing candidate information with immediate DB persistence.
- FR-A5: Admin must be able to delete a candidate, triggering a Linked List refresh.
- FR-A6: Admin must be able to search candidates by name (Binary Search) or by area (linear scan).
- FR-A7: Admin must be able to browse candidates sequentially using Next and Previous buttons (Linked List).
- FR-A8: Admin must be able to view all registered voters and approve pending verification requests.
- FR-A9: Admin must be able to view vote results, with candidates sorted in descending vote count order (Bubble Sort).

8.2 User Functions

- FR-U1: New users must be able to register by providing CNIC, name, email, password, and area.
- FR-U2: System must prevent unverified users from logging in.
- FR-U3: Verified users must be able to log in and access the User Dashboard.
- FR-U4: Users must be able to browse candidates via Next / Previous navigation (Linked List).
- FR-U5: Users must be able to search for a candidate by name or area.
- FR-U6: Each verified user may cast exactly one vote; duplicate votes must be prevented.
- FR-U7: Vote cast must increment the candidate's vote_count in MySQL and set has_voted = 1 for the voter.
- FR-U8: Users must be able to view the VoteProgress screen showing real-time vote percentages.

9. Non-Functional Requirements

Requirement	Description
Security	Only admin-verified users can cast votes. Passwords stored securely. Admin credentials protected in DB.
Performance	Binary Search provides $O(\log n)$ retrieval. System must handle 1000+ records without degradation.
Usability	Dark-themed Swing GUI with clear labels, status bars, and intuitive role-based navigation.
Reliability	Singleton DB connection with proper error handling (try-catch on every SQL operation).
Scalability	Linked List dynamically grows; MySQL scales to millions of records with indexing.
Maintainability	Modular package structure; DSA logic is fully decoupled from GUI and DB layers.
Portability	Java Swing runs on any JVM platform (Windows, macOS, Linux) without recompilation.

10. Tools and Technologies

Component	Technology	Notes
Programming Language	Java (JDK 17+)	Fully supports custom DSA and OOP
GUI Framework	Java Swing (javax.swing)	
IDE	IntelliJ IDEA	Project management, debugging, Git
Database	MySQL 8.x	Primary persistent data store

Component	Technology	Notes
DB Connectivity	JDBC MySQL Connector/J 8	Replaces C++ MySQL Connector
Version Control	Git / GitHub	4 active branches tracked
DSA — Linear DS	Custom CandidateLinkedList.java	Singly Linked List with circular nav
DSA — Search	Custom BinarySearch.java	Binary Search + Linear fallback
DSA — Sort 1	Custom VoteSorter.java (Bubble Sort)	Descending vote ranking
DSA — Sort 2	BinarySearch.sortByName (Insertion Sort)	Pre-sort for Binary Search

11. Version Control and Project Tracking

The project is maintained on GitHub under the repository DSA-Voting-System-Project. Each group member has a dedicated branch with regular commits reflecting their individual contributions:

Branch	Member	Primary Contribution
main	All Members	Merged, production-ready code
Fahad_Branch	Muhammad Fahad Khan	Admin Dashboard, DB connection, core DSA classes
Ahmad_Minhal	Ahmad Minhal	User Registration, UserLogin, UserConfirmation forms
Hamza_Branch	Ameer Hamza	UserDashboard, VoteProgress, vote casting logic

Regular commits are maintained in all branches to reflect continuous progress. The GitHub repository is used to monitor group contribution and track incremental development milestones.

<-----THE END----->